

Zero-Knowledge Proofs 応用サーベイ

詳解・翻訳ガイド

Lavin et al., “A Survey on the Applications of Zero-Knowledge Proofs” の日本語解説書

作成日: 2026-07-08

対象論文: Ryan Lavin, Xuekai Liu, Hardhik Mohanty, Logan Norman, Giovanni Zaarour, Bhaskar Krishnamachari, “A Survey on the Applications of Zero-Knowledge Proofs”, arXiv:2408.00243v2, 10 Apr 2026.

本書の位置付け: 原論文の構成、図表、主要な主張を保ちながら、自然な日本語に翻訳し、背景知識、実装上の読み方、批判的な注釈を加えた解説版である。固有名詞、論文内引用番号、システム名は原則として原文表記を維持した。

読み方の提案

ZKP の理論に詳しくない場合は、まず第 1 部の要約と用語集を読み、その後に第 2 部の基礎、最後に応用分野を読むと理解しやすい。実務者は、各章末の「実装上の読み方」と「設計判断の要点」を重点的に読むとよい。

目次（概略）

- 1. 本解説書の目的と全体要約
- 2. ZKP を読むための背景知識
- 3. 原論文 第1章 Introduction の翻訳と解説
- 4. 原論文 第2章 Foundations of Zero-Knowledge Proofs の翻訳と解説
- 5. 原論文 第3章 ZKP Software Tools and Platforms の翻訳と解説
- 6. 原論文 第4章 Blockchain Applications の翻訳と解説
- 7. 原論文 第5章 Non-Blockchain Applications の翻訳と解説
- 8. 原論文 第6章 Conclusion and Future Work の翻訳と解説
- 9. 横断的な設計論点、限界、今後の研究課題
- 10. 付録: 用語集、図表訳、参考文献ガイド

1. 本解説書の目的と全体要約

1.1 論文の目的

このサーベイ論文は、ゼロ知識証明（Zero-Knowledge Proofs, ZKPs）の応用分野を、理論だけでなく実装・デプロイ・開発ツールの観点から整理することを目的としている。特に、実用システムで多用される zkSNARKs を中心に置きつつ、zkSTARKs や Bulletproofs など他の証明ファミリとの違いも比較している。

原論文の特徴は、ZKP のアルゴリズムそのものを深掘りするだけでなく、ブロックチェーンのプライバシー、スケーリング、ストレージ、相互運用性、Proof of Reserves、さらに投票、認証、タイムロック、機械学習といった非ブロックチェーン応用までを、同じ評価軸で横断的に見る点にある。

1.2 論文の一文要約

一文要約

ZKP は「計算が正しかった」ことを、入力や中間過程を明かさずに、短く検証可能な形で証明する技術であり、現代の応用では「プライバシー」と「検証コスト削減」の二つの価値を軸に展開している。

1.3 本論文で一貫して使われる評価軸

原論文は、さまざまな ZKP 応用を比較する際、次の観点を重視している。

- 証明サイズ: ブロックチェーンではオンチェーン保存・検証コストに直結する。
- Prover time: 証明生成時間。ZK rollup や ZKML ではボトルネックになりやすい。
- Verifier time: 検証時間。オンチェーン検証ではガス代・処理遅延に直結する。
- メモリ使用量: 大規模トレース、FFT/NTT、MSM、ハッシュ多用型システムで重要。
- セットアップ仮定: trusted setup か transparent setup か。
- ブロックチェーン応用では、オンチェーン検証コストとデータ可用性（DA）コスト。

1.4 ZKP 用の二大値

第一の価値は succinctness（簡潔性）である。これは、巨大な計算を再実行しなくても、その計算が正しかったことを短い証明で検証できる性質を指す。たとえば rollup では、L2 で大量のトランザクションを処理し、L1 には圧縮されたデータと証明だけを提出する。

第二の価値は privacy（プライバシー）である。これは、証明の対象となる秘密データを明かさずに、条件を満たしている事実だけを示せる性質を指す。たとえば、残高が十分であること、ある資格を保有していること、ある匿名集合のメンバーであることを、具体的な残高・身元・履歴を明かさずに証明できる。

1.5 原論文の全構造

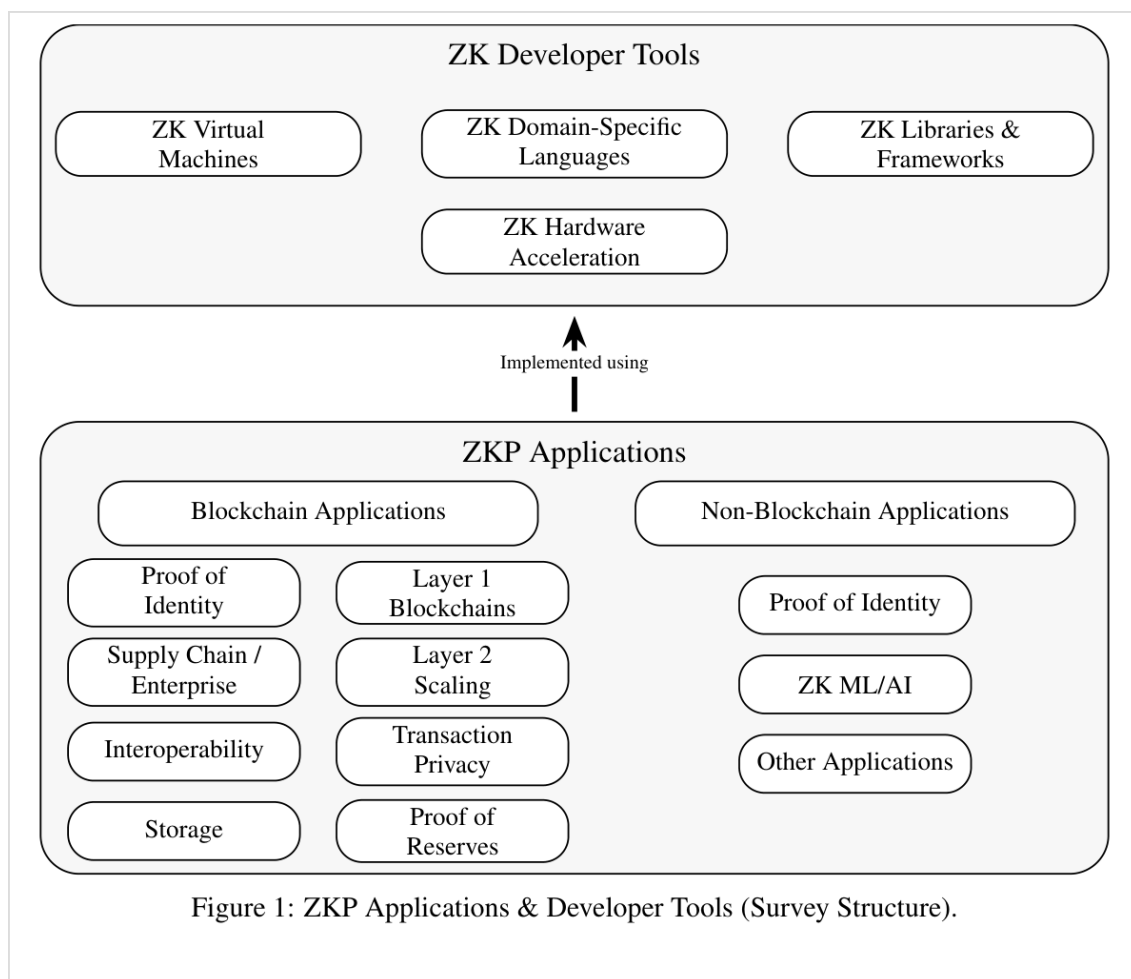


図 1（原論文 p.3）：ZKP 応用と開発ツールの全体構造。上段が開発基盤、下段が応用領域を表す。

図 1 の日本語ラベル訳

英語ラベル	日本語訳	意味
ZK Developer Tools	ZK 開発ツール	ZKP 応用を実装するための開発基盤
ZK Virtual Machines	ZK 仮想マシン	汎用プログラム実行を証明可能にする VM
ZK Domain-Specific Languages	ZK ドメイン固有言語	制約・回路生成に適した言語
ZK Libraries & Frameworks	ZK ライブラリとフレームワーク	ガジェット、曲線、証明バックエンド
ZK Hardware Acceleration	ZK ハードウェア高速化	GPU/FPGA/ASIC による MSM、FFT/NTT、ハッシュの高速化
ZKP Applications	ZKP 応用	ブロックチェーン応用と非ブロックチェーン応用
Implemented using	これらのツールで実装	応用は開発ツール群に支えられるという関係

図 1 は、論文全体の地図である。上段の開発ツール群は、ZKP を実装可能にするインフラを示す。下段の応用領域は、ブロックチェーン応用と非ブロックチェーン応用に分かれる。ブロックチェーン応用には、L1、L2、相互運用性、トランザクションプライバシー、ストレージ、Proof of Reserves、アイデンティティ、サプライチェーン/エンタープライズが含まれる。非ブロックチェーン応用には、ZKML/AI、アイデンティティ、その他の応用が含まれる。

2. ZKP を読むための背景知識

2.1 ゼロ知識証明とは何か

ゼロ知識証明は、「ある主張が真である」ことを、主張の裏側にある秘密情報を明かさずに検証者へ納得させる暗号プロトコルである。典型的には、証明者（prover）が秘密の witness を持ち、検証者（verifier）は公開された statement が真であるかどうかだけを知る。

たとえば「私はこのコミットメントの中身を知っている」「このトランザクションは残高を破壊していない」「このニューラルネットワークの推論結果は指定モデルから得られた」といった主張を、秘密の入力や中間状態を明かさずに示すことができる。

2.2 ZKP と混同しやすい周辺技術

プライバシー保護計算の比較

技術	主な目的	典型的な強み	典型的な弱み
ZKP	計算結果・条件充足の検証	短い公開検証、プライバシー、ブロックチェーンとの相性	証明生成が重い、回路設計が難しい
同型暗号（HE）	暗号化したまま計算	データを復号せず演算可能	計算コストが高い、検証性とは別問題
安全なマルチパーティ計算（MPC）	複数者が入力を隠したまま共同計算	共同計算に強い	通信量・ラウンド数・参加者管理が課題
TEE	ハードウェア隔離環境で実行	既存ソフトを動かしやすい	ハードウェアベンダやサイドチャンネルへの信頼が必要

2.3 statement、witness、relation

ZKP を理解するための最小単位は、statement、witness、relation の三つである。statement は公開される主張、witness は秘密の証拠、relation は「statement と witness が整合するか」を判定する規則である。ZKP は、witness を明かさずに relation が成り立つことを証明する。

例: 残高証明

statement: 「この口座の残高は 100 以上である」。witness: 実際の残高や署名鍵。relation: 署名が正しく、残高 ≥ 100 であること。ZKP は「100 以上である」事実だけを示し、残高そのものは隠す。

2.4 完全性、健全性、知識健全性、ゼロ知識性

完全性（completeness）：主張が真で、証明者と検証者が正しくプロトコルを実行すれば、検証者は受理する。

健全性（soundness）：主張が偽であれば、効率的な不正証明者は検証者をだませない。SNARK では多くの場合、計算量仮定に基づく computational soundness である。

知識健全性（knowledge soundness）：証明者が検証者を納得させられるなら、その証明者は有効な witness を知っている、と形式化できる。

ゼロ知識性（zero knowledge）：証明から witness について何も漏れない。検証者が学ぶのは statement が真である事実だけである。

2.5 trusted setup と transparent setup

多くの zkSNARK では、証明者と検証者が共有する公開パラメータ、すなわち Common Reference String (CRS) が必要になる。CRS 生成時に秘密の trapdoor が使われ、その秘密が漏れると証明偽造につながる場合がある。このため、複数者が参加するセレモニーで「少なくとも一人が正直に秘密を破棄すれば安全」という形で信頼を分散する。

一方、transparent setup を採る STARK 系や FRI 系では、秘密の trapdoor に依存せず、公開ランダム性や標準的仮定からパラメータを生成する。この違いは、規制分野、公共システム、長期保存が必要な証明で重要になる。

2.6 Fiat-Shamir 変換

Fiat-Shamir 変換は、対話型プロトコルで検証者が送るランダムチャレンジを、ハッシュ関数の出力で置き換えることで非対話化する技法である。非対話化は、ブロックチェーンのように「証明を提出すれば誰でも後から検証できる」環境で不可欠である。ただし、非対話性と trusted setup は別概念である。非対話でも trusted setup が必要な方式は存在する。

2.7 なぜ有限体・多項式が出てくるのか

現代の ZKP、特に SNARK/STARK は、計算を有限体上の代数的制約へ変換する。理由は、巨大な計算の正しさを多項式恒等式や多項式評価の問題へ還元できるからである。計算を「制約の集合」として表すと、証明者は「すべての制約を満たす witness を知っている」ことを短く示せる。

2.8 務上のキワド

実務で頻出する ZKP 用語

用語	意味	実務上の注意
MSM	multi-scalar multiplication。楕円曲線の線形結合計算	SNARK の証明生成で最重要ボトルネックの一つ
FFT/NTT	多項式評価・補間を高速化する変換	大規模回路ではメモリ帯域が効く
FRI	STARK 系で使われる低次数性検査	透明セットアップ・ポスト量子性に関係
PCS	Polynomial Commitment Scheme	多項式を短くコミットし、評価値を開示できる
lookup	表引き制約を効率化する議論	ハッシュ、レンジチェック、非ネイティブ演算を安くする
recursion	証明の検証をさらに証明すること	ロールアップ、IVC、長期計算の合成に重要

3. 原論文 第 1 章 Introduction の翻訳と解説

3.1 Abstract の翻訳

ゼロ知識証明 (ZKPs) は、一方の当事者がある statement の真実性を、基礎データを明かすことなく証明できるようにすることで、計算の完全性とプライバシーを可能にする。同型暗号や安全なマルチパーティ計算といった代替技術と比較すると、ZKP は汎用性と信頼仮定の少なさという点で独自の利点を持ち、ブロックチェーンシステムから計算タスクの秘密検証まで幅広い応用を持つ。

本サーベイは、ZKP の技術的概観を提供し、特に zkSNARKs という実用上重要性を増している部分集合に焦点を当てる。従来のサーベイがアルゴリズムや理論面に重点を置くのに対し、本論文は実デプロイと最近のユースケースをより広く扱う。対象領域には、ブロックチェーンのプライバシー、スケーリング、ストレージ、相互運用性、さらに投票、認証、タイムロック、機械学習などの非ブロックチェーン応用が含まれる。

一貫した比較のために、本論文は、応用領域の分類、証明サイズ・証明者時間・検証者時間・メモリ・セットアップ仮定を含む評価基準、主要トレードオフと代表システムをまとめた比較表を提示する。また、ゼロ知識仮想マシン、ドメイン固有言語、ライブラリ、フレームワークといった支援インフラも扱う。実用システムで普及していることから zkSNARKs に重点を置きつつ、透明性と性能のトレードオフを明確化するため、zkSTARKs や Bulletproofs とも比較する。

3.2 Introduction 本文の解説

ZKP の起源は、Goldwasser、Micali、Rackoff による 1980 年代の seminal work にさかのぼる。この研究では、証明者から検証者へ移る情報量を測る指標として knowledge complexity が導入された。その後、Goldreich らの研究により、ZKP の対象はより広い計算問題へ拡張され、特定の暗号仮定のもとで NP 完全問題にも適用できることが示された。

次の大きな転換点は succinctness、すなわち「短い証明で巨大計算を検証できる」性質の導入である。Kilian の設計パラダイムを経て、SNARK/zkSNARK の簡潔性と非対話性は、ブロックチェーンや非ブロックチェーンの実用システムへの適用可能性を大きく高めた。

論文は読者として、(i) システム・セキュリティ研究者、(ii) ブロックチェーンエンジニア、(iii) 学生および応用暗号の実務者を想定している。研究者にとっては応用地図とボトルネックの把握、エンジニアにとってはスケーラビリティ・プライバシー・コスト設計の指針、学習者にとっては基礎部品から現代ツールチェーンまでの案内図として機能する。

現代のデジタルシステムには、透明性とプライバシーの緊張関係がある。ブロックチェーンは不正防止のため透明性を重視するが、その透明性はユーザの取引履歴や行動パターンを露出させる。擬名性だけでは十分ではなく、オンチェーンデータとオフチェーンデータの相関解析により匿名性が破れる可能性がある。この問題に対して ZKP は、検証可能性を保ちながらデータを秘匿する手段を提供する。

ZKP の実用的価値は verifiable computation に基づく。すなわち、計算を実行したこと、またその結果が正しいことを、入力や計算過程を明かさずに証明する能力である。この能力が、簡潔性とプライバシーという二つの価値を生む。

背景補足: なぜブロックチェーンで ZKP が急速に普及したか

ブロックチェーンでは全ノードが状態遷移を検証する必要があるため、計算とデータが高コストである。ZKP は「全員が再計算する」モデルを「少数の証明者が計算し、全員が短い証明を検証する」モデルへ変える。この構造が rollup、zkEVM、ストレージ証明、相互運用性で強力に働く。

3.3 先行サーベイとの差分

原論文は、Morais et al. のゼロ知識レンジ証明サーベイ、および Sun et al. のブロックチェーンにおける ZKP サーベイと比較して、自らの位置付けを説明している。Morais らは ZK range proof のアルゴリズムの詳細とユースケースに焦点を当てる。一方、本論文はより幅広い応用カテゴリ、ツール、インフラを扱う。Sun らは ZKP とブロックチェーンの交差領域に焦点を当てるが、本論文は非ブロックチェーン応用も含める。

3.4 Introduction の要点

- ZKP の本質は「検証可能性」と「秘匿性」の両立である。
- 実用化を進めたのは、zkSNARKs の簡潔性と非対話性である。
- ZKP の応用はブロックチェーンに偏りがちだが、本人確認、投票、機械学習、メディア認証などにも拡大している。
- 性能評価では、証明サイズだけでなく、証明生成時間、検証時間、メモリ、セットアップ仮定、オンチェーンコストを見る必要がある。

4. 原論文 第 2 章 Foundations of Zero-Knowledge Proofs の翻訳と解説

4.1 ZKP、SNARK、zkSNARK の層構造

第 2 章は、ZKP の基礎概念を定義し、ZKP という広いクラスと、SNARK、zkSNARK という実用上重要な精緻化との関係を整理する。原論文の重要な注意点は、「SNARK は必ずしも zero-knowledge ではない」という点である。SNARK は succinct non-interactive argument of knowledge であり、ゼロ知識性は別途満たす必要がある。

表 1 の翻訳: 実務で使われる代表的証明ファミリの大まかな傾向

カテゴリ	ゼロ知識	簡潔	対話性	セットアップ	ポスト量子性
ZKP 一般	はい	任意	任意	任意	方式による
SNARK	任意	はい	非対話	多くは trusted	通常はいいいえ
zkSNARK	はい	はい	非対話	多くは trusted	通常はいいいえ
zkSTARK	はい	はい	非対話	transparent	はい
Bulletproofs	はい	いいえ	非対話	transparent	通常はいいいえ

この表は「大まかな傾向」であり、すべての方式に普遍的に当てはまるわけではない。近年は透明セットアップの SNARK、ポスト量子性を意識した SNARK、より小さい STARK 証明など、境界を越える研究も進んでいる。

4.2 証明システムの基本性質

原論文は、完全性、計算的健全性、知識健全性、ゼロ知識性を定義する。ここで「argument」という語は、情報理論的に無制限な不正者に対する健全性ではなく、効率的な不正者に対して計算量仮定のもとで安全であることを表す。SNARK は argument であり、標準的暗号仮定に依存する。

4.3 CRS、trusted setup、transparent setup

多くのデプロイ済み zkSNARK では、証明者と検証者が共有する public parameters が必要となる。CRS が trapdoor randomness とともに生成される場合、その trapdoor が漏れると、証明偽造や soundness 破壊につながる可能性がある。そのため、複数参加者のセレモニーを用いて、少なくとも一人が正直で秘密を破棄すれば安全という設計が採られる。

transparent setup は、この toxic waste への依存を避ける設計である。STARK 系は代表例であり、公開ランダム性とハッシュベースの構成により、trusted setup に対する懸念を低減する。

4.4 ZKP パイプライン: Python から多項式へ

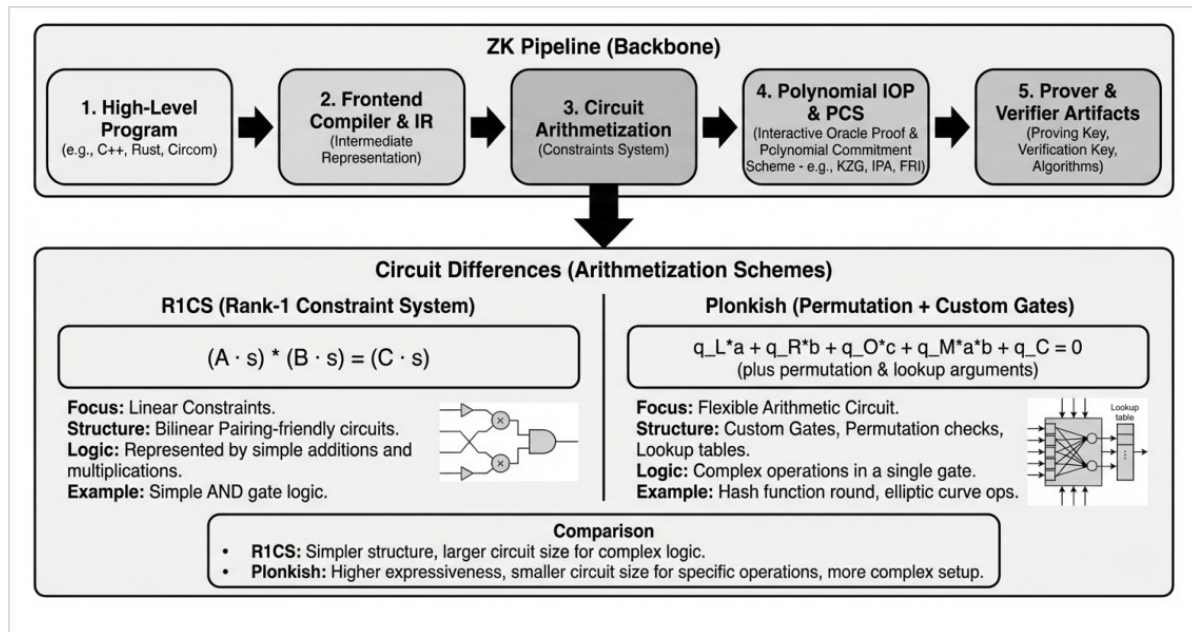


図2 (原論文 p.5) : ZKP パイプライン。高水準プログラムから制約、多項式 IOP/PCS、証明・検証アーティファクトへ進む。

図2の主要ラベル記

英語ラベル	日本語訳	説明
High-Level Program	高水準プログラム	C++, Rust, Circom など書かれる元の計算
Frontend Compiler & IR	フロントエンドコンパイラと中間表現	プログラムを制約生成しやすい表現へ変換
Circuit Arithmetization	回路の算術化	計算を R1CS, Plonkish, AIR などへ符号化
Polynomial IOP & PCS	多項式 IOP と多項式コミットメント	制約充足性を多項式関係へ落とし込む
Prover & Verifier Artifacts	証明者・検証者アーティファクト	証明鍵、検証鍵、アルゴリズム、証明

4.4.1 フロントエンド: 高水準コードから回路へ

原論文の例では、三変数多項式 $f(a, b, c) = a^2 b^2 + b^2 c^2$ の評価を考える。一般のプログラムを ZKP に載せるには、加算や乗算などの単純な算術操作に分解し、有限体 F_p 上の演算として表現する。

有限体を使うのは、加算・乗算・逆元などが閉じた代数構造を持ち、多項式を用いた検査と相性が良いためである。ZKP では、通常の整数やビット列を有限体要素として扱うため、レンジチェックやビット分解が重要なコスト要因になる。

4.4.2 算術化: 回路から行列へ

算術化とは、回路や計算モデルの配線・ゲート演算を、証明生成に適した代数表現へエンコードする過程である。原論文は代表例として R1CS を扱う。R1CS では、計算を次の形の制約集合として表す。

$$(A \cdot w) \cdot (B \cdot w) = (C \cdot w)$$

ここで A、B、C は行列、w は witness ベクトル、 $\circ$ は要素ごとの積を表す。各制約行は、回路の一つのゲートや中間計算に対応する。例の多項式では、a、b、c の二乗、二つの交差乗算、最後の加算により、複数の制約が生じる。

witness ベクトルの例は、 $w = [1, a, b, c, a^2, b^2, c^2, a^2 b^2, b^2 c^2, \text{result}]$ のように、入力と中間値と出力を含む。witness をそのまま公開すれば秘密が漏れるため、ZKP は「この witness が制約を満たす」ことだけを隠して証明する。

4.4.3 バックエンド: 行列から多項式へ

バックエンドでは、R1CS 制約を Quadratic Arithmetic Program (QAP) などの多項式関係へ変換する。原論文では次の形が示される。

$A(x) \cdot B(x) - C(x) = h(x) \cdot Z(x)$

$Z(x)$ は制約インデックス集合で消える target polynomial、 $h(x)$ は制約充足性をエンコードする多項式である。証明者は「そのような $h(x)$ が存在する」ことを短い証明に落とし込む。検証者は多項式関係の一部を効率的にチェックするだけでよく、計算全体を再実行しない。

表 2 の翻訳: 算術化戦略の比較

性質	R1CS	Plonkish	AIR	CCS (folding)
制約スタイル	線形結合に対する双線形制約	カスタムゲート + permutation/copy 制約	実行トレース上の多項式制約	folding/IVC 向けに一般化された制約
強み	単純で成熟したツール	柔軟な算術化、効率的 lookup	transparent、巨大トレースに向く	効率的 folding と incremental verification
典型的な prover コスト要因	MSM。一部方式ではペアリングも関与	FFT/NTT + MSM	ハッシュ、大規模トレースのメモリ/IO	バックエンド依存
lookup	通常はガジェットで模倣	Plookup/LogUp/Lasso などネイティブ	LogUp などの lookup argument	folded system 内の構成による
再帰/IVC	pairing cycle ベースが多い	accumulation/recursion-friendly 設計	FRI スタイルの再帰・合成	Nova、HyperNova などの folding
例	Groth16、Marlin	Plonk、Halo2	STARKs、Cairo	Nova、HyperNova

4.5 情報理論と暗の接合

第 2 章の最後は、対話型証明、情報理論的健全性、暗号的部品の関係を説明する。現代 SNARK の典型的設計は、計算を多項式チェックへ還元する PIOP/IOP、巨大多項式に短くコミットする PCS、そして Fiat-Shamir による非対話を組み合わせる。

この組み合わせにより、公開検証可能で、短い証明を持ち、場合によっては witness を秘匿する証明が得られる。ただし、セットアップ仮定、ハッシュ関数のランダムオラクルモデル、ペアリング仮定、透明性、ポスト量子性など、方式ごとに異なる仮定が入る。

設計判断の要点

理論上「ZKP」と呼ばれていても、実装では「何を証明するか」「どの算術化を使うか」「どの PCS を使うか」「trusted setup を許すか」「オンチェーン検証するか」でまったく違うシステムになる。

5. 原論文 第 3 章 ZKP Software Tools and Platforms の翻訳と解説

5.1 ツルチェン分類

第 3 章は、ZKP アプリケーションを支えるソフトウェア基盤を、(i) zkVM、(ii) zkDSL・回路言語、(iii) ライブラリ・フレームワーク、(iv) 証明バックエンド、(v) ハードウェアアクセラレータに分類する。

5.2 Zero-Knowledge Virtual Machines (zkVM)

5.2.1 動機と定義

zkVM は、任意の計算を実行し、その実行が正しかったことを効率的に検証可能なゼロ知識証明として生成する仮想マシンである。プログラムやデータのプライバシーを保ちながら、VM の実行そのものを証明対象にする。

従来、ZKP アプリケーションを作るには、回路設計、暗号プリミティブ、証明バックエンドの知識が必要だった。zkVM はこの複雑性を抽象化し、Rust や C++ のような既存言語で書いたプログラムを証明可能にする方向へ進んでいる。

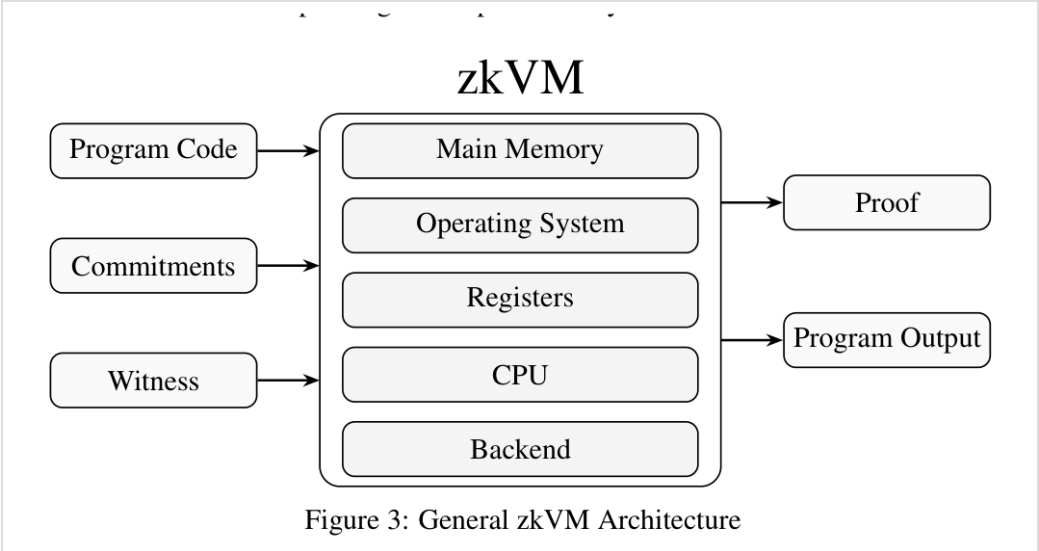


Figure 3: General zkVM Architecture

図 3 (原論文 p.7) : 一般的な zkVM アーキテクチャ。左からプログラム・コミットメント・witness が入力され、VM 内部で実行され、証明と出力が得られる。

図 3 のラベル訳

英語ラベル	日本語訳	意味
Program Code	プログラムコード	証明対象となる実行プログラム
Commitments	コミットメント	公開アーティファクトや状態根などへの束縛
Witness	witness/秘密実行データ	秘密入力や中間実行情報
Main Memory	主記憶	VM の仮想メモリ
Operating System	OS/ランタイム	実行環境の管理
Registers	レジスタ	CPU 状態
CPU	CPU/ISA インタープリタ	命令実行
Backend	証明バックエンド	実行トレースを証明へ変換

英語ラベル	日本語訳	意味
Proof	証明	検証者が確認する短い証明
Program Output	プログラム出力	公開または検証対象の出力

5.2.2 方法論

zkVM では、入力プログラムが RISC-V のような汎用 ISA、または Miden Assembly のような ZK 向け ISA にコンパイルされる。実行トレースが生成され、命令の正しさ、メモリ整合性、ハッシュ、コミットメント開示、制約チェックなどが証明される。

性能上のボトルネックは、概念上の VM ブロックそのものよりも、トレース生成と証明バックエンドのカーネルに集中する。具体的には、ISA セマンティクス制約、メモリ一貫性、ハッシュ、コミットメント、MSM、FFT/NTT などである。

ZK 計算で共通するオーバーヘッドとして、レンジチェック、ビット演算、ハッシュが挙げられる。SHA-256 のような通常のハッシュはビット操作が多く、有限体上では高コストになりやすい。このため Poseidon、Rescue、MiMC などの SNARK-friendly hash が使われる。

表 3 の翻訳: 代表的 zkVM の設計選択

zkVM	実行エンコード	証明ファミリ	ハッシュ戦略	再帰/合成
RISC Zero	RISC-V トレース + 制約	STARK。必要に応じ SNARK wrap	SHA-2 などハッシュ多用	対応
CairoVM	AIR (トレース制約)	STARK	ハッシュ多用トレース	対応
Miden VM	AIR (スタックトレース)	STARK	ZK-friendly permutation hash	対応

5.2.3 プロジェクト

原論文は、TinyRAM、Hawk、Zinc、RISC Zero、CairoVM、Leo、Miden、Triton、OlaVM、Powdr、Jolt、SP1、Nexus、Validaなどを挙げる。代表例として、RISC Zero は Rust/C++ コードを組み込み RISC-V マイクロプロセッサ上で正しく実行したことを示す。CairoVM は STARK ベースの von Neumann アーキテクチャを実装し、Starknet の基盤になっている。Miden VM は Ethereum を拡張する ZK rollup 向け VM で、並列実行やクライアント側証明を意識する。

背景補足: zkVM と専用回路の違い

専用回路は特定タスクに最適化できるため効率が高いことが多い。一方、zkVM は汎用プログラムを証明しやすく、開発者体験を改善する。ただし、汎用性のために命令実行・メモリ・制御フローを証明するコストが入る。

5.3 Domain Specific Languages (zkDSL)

5.3.1 動機と定義

zkDSL は、高水準プログラムと低水準の ZK 回路記述を橋渡しする言語である。目的は、高水準言語を算術回路へ変換し、証明システムでプログラム実行の証明を生成できるようにすることである。

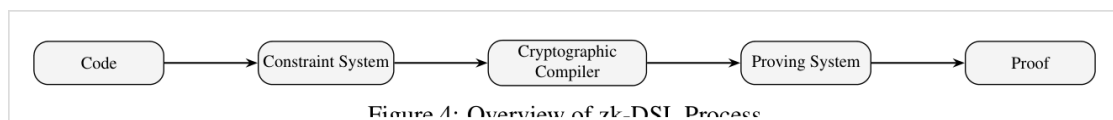


図4 (原論文 p.9) : zkDSL の処理概要。コードから制約システム、暗号コンパイラ、証明システムを経て証明へ進む。

表4の翻訳: zkDSL の比較

zkDSL	証明システム	中間表現	構文の雰囲気	用途
O1JS/Snarky	Kimchi Plonk variant	Plonk variant	TypeScript	汎用、スマートコントラクト
Circom	Groth16	R1CS	N/A	回路開発
Cairo	STARK	AIR	Rust 風	スマートコントラクト
Noir	Plonk、Groth16、Marlin	ACIR	Rust 風	回路開発
Juvix	Plonk、Halo2	Vampire	OCaml 風	Anoma intents
Lurk	Groth16、Nova	R1CS	Common Lisp/Scheme	汎用

5.3.2 方法論とバグの種類

zkDSL は抽象化を提供するが、通常のプログラミング言語とは異なる難しさがある。メモリ管理、再帰、条件分岐、可変変数、ユーザ定義構造などを制約システムに落とす際に制約数や安全性の問題が生じる。

実務で特に危険なのは under-constrained circuit である。これは、意図した関係を制約が完全に表しておらず、不正な witness でも制約を満たしてしまう状態を指す。その他、ガジェットのエッジケース漏れ、レンジチェック不足、witness malleability など重要である。

対策として、単体テスト、property-based testing、fuzzing、バックエンド間の differential testing、監査チェックリストが使われる。ZK の理論的健全性は「制約が正しく書かれている」ことを前提にするため、回路実装の検証は極めて重要である。

5.3.3 プロジェクト

Circom は Tornado Cash や Dark Forest など使われた回路記述言語であり、R1CS へのコンパイルを行う。Circomlib は再利用可能な回路テンプレートを提供する。Lurk は LISP ベースの Turing 完全な言語で、任意プログラムの zkSNARK 生成を目指す。Leo は Aleo 向けのゼロ知識アプリケーション言語で、形式検証と succinct verification を重視する。

5.4 Libraries and Frameworks

5.4.1 動機と定義

ZKP 開発には、有限体、楕円曲線、暗号プリミティブ、証明システム、制約ガジェットの知識が必要である。ライブラリとフレームワークは、これらを再利用可能な部品として提供し、開発者が高水準の設計に集中できるようにする。

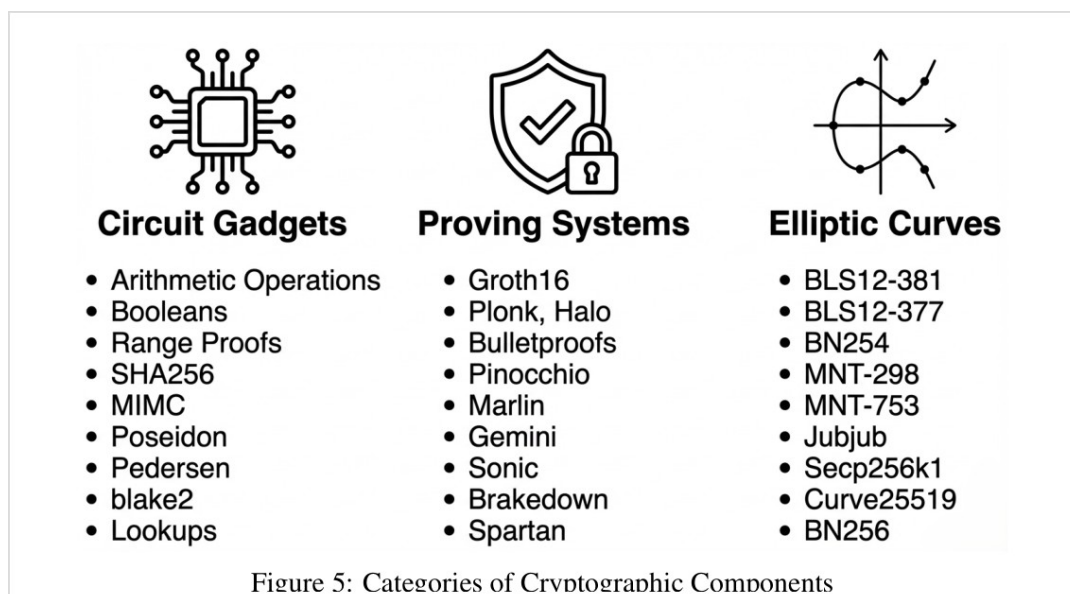


図5（原論文 p.10）：暗号コンポーネントの分類。Circuit Gadgets、Proving Systems、Elliptic Curves に分かれる。

図5の要素記

カテゴリ	代表例	役割
Circuit Gadgets	算術演算、ブール、レンジ証明、SHA256、MiMC、Poseidon、Pedersen、blake2、lookup	回路内で再利用する部品
Proving Systems	Groth16、Plonk/Halo、Bulletproofs、Pinocchio、Marlin、Gemini、Sonic、Brakedown、Spartan	具体的な証明生成・検証方式
Elliptic Curves	BLS12-381、BLS12-377、BN254、MNT、Jubjub、secp256k1、Curve25519、BN256	ペアリング、署名、コミットメントなどの基盤

5.4.2 方法論

フレームワーク設計では、小さい有限体を使うこと、field selection の柔軟性、安価な暗号プリミティブの採用が重要になる。ethSTARK や Plonky2 のようなシステムでは、64-bit 程度の小さい素体を使うことで CPU キャッシュ効率や proving performance を高める設計が見られる。

SNARK では、SHA-2 や ECDSA のような従来プリミティブが非ネイティブフィールド演算になり、高コスト化しやすい。そのため、Poseidon、Keccak、Pedersen、MiMC、EdDSA など、ZK に比較的適した代替が用いられる。

大きな高速化テーマには、Pippenger 法などによる MSM 改善、FFT/NTT のチューニング、lookup argument と custom gate、再帰と proof composition、GPU/FPGA/ASIC による hardware acceleration がある。

表5の翻訳: ZK ライブラリで使われる主要楕円曲線

ライブラリ	BN254	BLS12-381	BLS12-377
arkworks	対応	対応	対応
gnark	対応	対応	対応
blstrs		対応	
ffjavascript	対応	対応	

ライブラリ	BN254	BLS12-381	BLS12-377
pairing_ce	対応	対応	
zkcrypto		対応	
halo2_curves	対応		
pasta_curves		対応相当 (Pasta 系)	

5.4.3 プロジェクト

Arkworks は Rust の zkSNARK エコシステムで、有限体、R1CS、楕円曲線などを提供する。Gnark は Go で回路設計を行うライブラリで、Groth16 と Plonk、複数の楕円曲線をサポートする。CirC は暗号システムと検証のためのコンパイラインフラで、高水準プログラムを算術制約へ変換する。

O1JS は Mina の zkApps 向け TypeScript ライブラリであり、Web 開発者に親しみやすい形で ZK プログラムとスマートコントラクトを書ける。Halo2 は Zcash 由来の proving system で、lookup や permutation などの構成要素を持つ。Nova は folding scheme に基づく recursive SNARK で、incrementally verifiable computation (IVC) を可能にする。

5.5 Hardware Acceleration

5.5.1 動機と定義

ZK 証明生成は歴史的に遅く、メモリ要求も大きかった。証明生成で重い計算、特に MSM、FFT/NTT、ハッシュを高速化するため、GPU、FPGA、ASIC が使われる。

5.5.2 方法論

ハードウェア高速化の制約は、メモリ容量、メモリアクセス速度、データ転送速度、算術ユニット速度である。MSM は並列化しやすいが、巨大ベクトルではメモリ使用量とスケーラビリティが課題になる。NTT は頻繁なデータシャッフルがあるため、分散や GPU/CPU 間転送が難しくなる。

表 6 の翻訳: ZK 主要カーネルのハードウェア高速化比較

カーネル/制約	CPU	GPU	FPGA	ASIC
MSM	柔軟なベースライン	強い並列性	低遅延パイプライン	最高スループットの可能性
FFT/NTT	最適化ライブラリ	帯域が合えば良好	ストリーミングに合えば強い	専用化で強い
Hashing/Merkle	ベースライン	バッチハッシュに強い	パイプラインに非常に強い	固定ハッシュに非常に強い
Memory bandwidth	キャッシュ制約	HBM は有利、PCIe 転送は不利	オンチップ帯域が有利	設計依存

5.5.3 プロジェクト

Ingonyama は GPU 向け暗号ライブラリ ICICLE を提供し、楕円曲線演算、MSM、NTT、Poseidon hash などを実装する。Cysic は ASIC と zkVM 高速化を扱う。Fabric Cryptography は ZKP、FHE、MPC などに向けた Verifiable Processing Unit (VPU) を提案する。Irreducible は FPGA アクセラレーションを用いた proof-as-a-service、Supranational は BLS12-381 などの高速暗号実装を提供する。

実装上の読み方

ZKP システムの性能は、証明方式の理論計算量だけでは決まらない。どのカーネルが支配的か、GPU に乗るか、メモリ帯域が足りるか、データ可用性コストが支配するかをアプリケーションごとに見る必要がある。

6. 原論文 第 4 章 Blockchain Applications の翻訳と解説

6.1 ブロックチェーン用の全像

表 7 の翻訳: ブロックチェーンベース ZKP 応用の概観

ユースケース	代表プロジェクト
Layer 1 Blockchains	Zcash、Aleo、Mina
Layer 2 Scaling	Polygon zkEVM、zkSync Era、Scroll、Linea、Starknet、Aztec
Smart Contract/Transaction Privacy	Hawk、Tornado Cash、Privacy Pools、Penumbra、Mina zkApps、Noir
Proof of Identity	Semaphore、WorldID、zPass、Galxe protocol
Supply Chain/Enterprise Blockchain	QEDIT、zk-BeSC、Sahai et al.
Interoperability	zkBridge、Telepathy
Blockchain Storage	Herodotus、Filecoin
Proof of Reserves	Provisions、Proven、ZeKnow Solv

6.2 Layer 1 Blockchains

6.2.1 動機と定義

Ethereum や Bitcoin のような L1 ブロックチェーンは、透明で改ざん困難な台帳を提供する。しかし、この透明性はプライバシーとスケーラビリティの課題を生む。ZKP ベースの L1 は、トランザクション秘匿と状態圧縮をベースレイヤに組み込み、透明性、プライバシー、データ効率を両立しようとする。

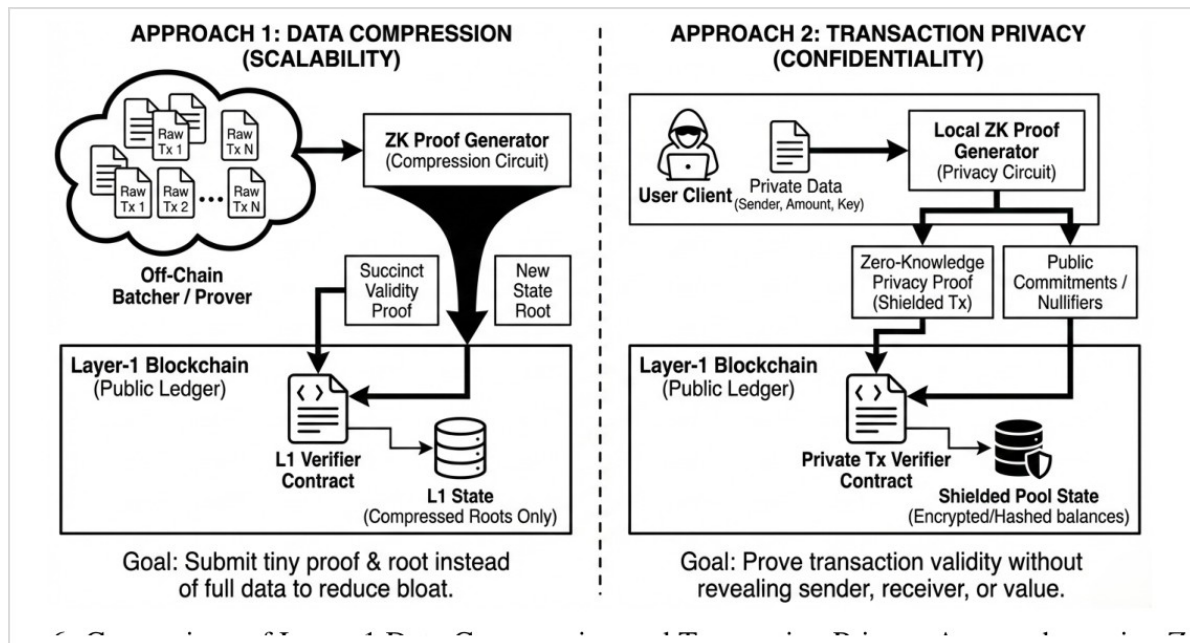


図 6 (原論文 p.13) : L1 における ZKP の二つの利用法。左はデータ圧縮/スケーラビリティ、右はトランザクションプライバシー。

図 6 のラベル訳

英語ラベル	日本語訳	意味
Approach 1: Data Compression (Scalability)	アプローチ 1: データ圧縮 (スケーラビリティ)	大量データの代わりに短い妥当性証明と状態根を提出

英語ラベル	日本語訳	意味
Off-Chain Batchers / Prover	オフチェーン batcher/prover	トランザクションをまとめて証明を生成
Succinct Validity Proof	短い妥当性証明	L1 で検証する証明
New State Root	新しい状態根	圧縮された状態更新のコミットメント
Approach 2: Transaction Privacy (Confidentiality)	アプローチ 2: 取引プライバシー (機密性)	送信者・受信者・金額を明かさず妥当性を証明
Public Commitments / Nullifiers	公開コミットメント/ヌリファイア	二重支出防止と匿名性のための公開値
Shielded Pool State	シールドプール状態	暗号化/ハッシュ化された残高状態

6.2.2 方法論

L1 ブロックチェーンは、zkSNARKs や zkSTARKs を用いて、送信者、受信者、金額などの重要情報を隠したまま、トランザクションの正当性を検証する。また、状態圧縮により、大量のトランザクションや状態遷移を短い証明にまとめ、オンチェーンデータ量を減らす。

ただし、UX、証明遅延、鍵管理、回路アップグレード、検証鍵、セットアップ成果物の管理などの運用複雑性が増す。

6.2.3 応用

Zcash は、zkSNARKs を用いて sender、receiver、amount を秘匿する shielded transaction を実現した初期のプライバシー重視 L1 である。オプションとして透明トランザクションとシールドトランザクションを選べる。

Aleo は、zkSNARK ベースのフレームワークにより、プライベートかつ検証可能な dApps を構築する L1 である。Mina Protocol は、recursive zkSNARKs により、処理済みトランザクション数にかかわらずチェーンの妥当性証明を小さく保つことを目指す。

6.3 Layer 2 Scaling

6.3.1 動機と定義

ブロックチェーンの大きな制約はブロックスペースの希少性とコストである。ZK rollup、より正確には validity rollup は、L2 で大量の計算を行い、L1 には状態遷移の正当性を示す短い証明を提出する。ここで zero-knowledge privacy は必須ではなく、プロトコル固有の選択である。

6.3.2 方法論

L2 rollup では、重い計算はオフチェーンで行われ、状態根、deposit、withdrawal、圧縮されたトランザクションデータ、証明が L1 の rollup contract に提出される。sequencer はトランザクションを順序付け、L2 ブロックを作成し、バッチと validity proof を L1 へ投稿する。

L1 contract は旧状態根と新状態根を比較し、証明が有効であれば状態を更新する。データは calldata または EIP-4844 後の blob に書き込まれる。ユーザ手数料は、証明検証コストとデータ可用性コストに大きく左右される。

zkEVM は EVM の opcodes を ZK 回路で再現し、Ethereum 互換性を保ちながら状態遷移を証明する。Custom VM は ZK に適した新 VM/言語を使うため効率や設計自由度が高いが、開発者は新しい言語やツールを学ぶ必要がある。

表 8 の翻訳: 代表的 L2 validity systems の比較

L2	証明ファミリ	セットアップ	EVM レベル	言語/ツール	DA モード	証明スループットの目安
zkSync Era	SNARK	trusted	EVM-compatible	Solidity/Vyper + custom compiler	rollup/validium	中
Polygon zkEVM	SNARK	trusted	EVM-equivalent	Solidity. custom compiler なし	rollup/validium	中
Scroll	SNARK	trusted	EVM-compatible	Solidity. 変更は小さい	rollup	中
Linea	SNARK	trusted	EVM-equivalent	Solidity. 標準 tooling	rollup	中
StarkNet	STARK	transparent	custom VM	Cairo toolchain	rollup/validium/volition	高
Aztec	SNARK	trusted	custom VM	Noir/ACIR toolchain	rollup	低～中

6.3.3 応用

zkSync Era は Matter Labs による zkEVM rollup で、zkSNARKs によりオフチェーン計算の validity proof を提供する。Solidity/Vyper から custom compiler を通じて zkEVM に適した bytecode を生成する。native account abstraction も特徴である。

Polygon zkEVM は EVM-equivalence を目指し、別コンパイラなしに既存 Solidity contract をデプロイしやすくする。Polygon CDK は application-specific chains を validium としてデプロイする枠組みを提供する。

Scroll と Linea は、Ethereum 開発体験を維持しながら ZK rollup に適応する設計を採る。StarkNet は STARK と Cairo を用いる custom VM 系であり、Cairo -> Sierra -> Casm というコンパイル経路を持つ。StarkEx は特定ユースケース向けの L2 scalability service で、ZK-Rollup、Validium、Volition の DA モードを選べる。

Aztec は Noir による private smart contracts を重視し、public/private state を扱う。

6.4 Blockchain Interoperability

6.4.1 動機と定義

複数の L1、L2、モジュラーチェーンが並立する現在、資産移転、メッセージパッシング、データ参照を安全に行う相互運用性が必要である。ZKP は、一方のチェーンで起きた状態変化やブロック実行を、別のチェーン上で短く検証する手段を提供する。

6.4.2 方法論

典型的には、middleware が送信元チェーンのブロックヘッダや状態変化の correctness proof を生成し、受信チェーン上の smart contract がその証明を検証してアプリケーションロジックを実行する。これにより、ブリッジ、クロスチェーン DAO 投票、NFT 移転などが可能になる。

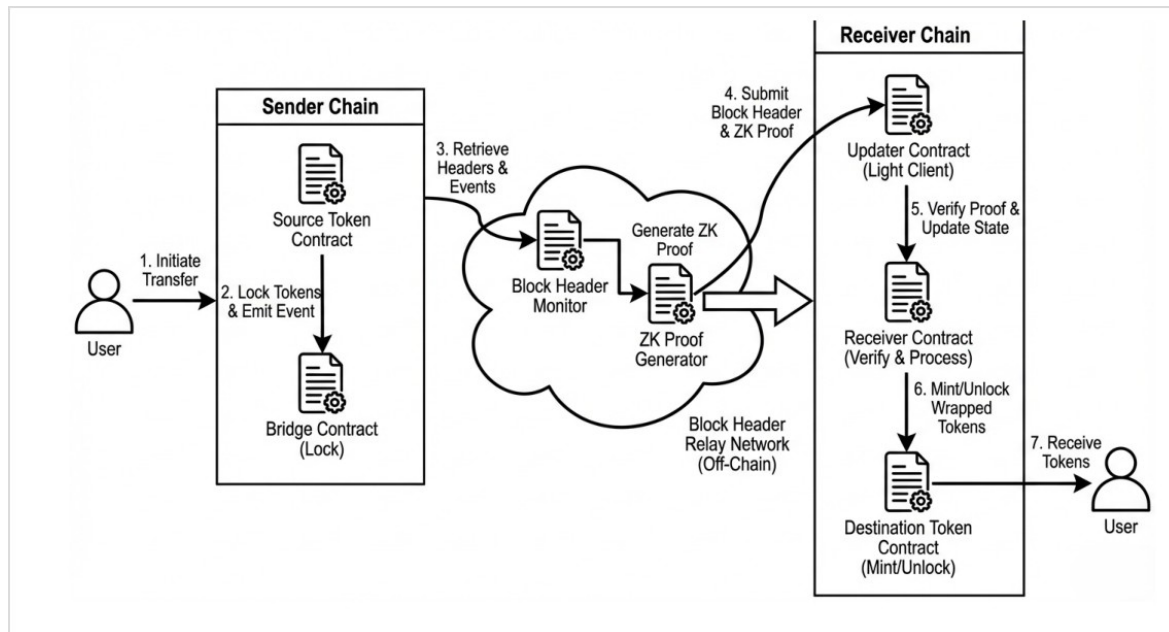


図7（原論文 p.17）：zkBridge によるクロスチェーントークン転送のアーキテクチャ。

図7のラベル訳

英語ラベル	日本語訳	意味
Sender Chain	送信元チェーン	元の資産が存在するチェーン
Receiver Chain	受信先チェーン	ラップされた資産を受け取るチェーン
Bridge Contract (Lock)	ブリッジコントラクト（ロック）	送信元で資産をロック
Block Header Monitor	ブロックヘッダ監視	ヘッダとイベントを取得
ZK Proof Generator	ZK 証明生成器	ヘッダ妥当性などを証明
Updater Contract (Light Client)	更新コントラクト（ライトクライアント）	受信先でヘッダリストを更新
Receiver Contract (Verify & Process)	受信コントラクト（検証・処理）	アプリケーション固有処理を実行
Destination Token Contract (Mint/Unlock)	受信先トークンコントラクト（mint/unlock）	資産を発行または解放

6.4.3 応用

zkBridge は、アプリケーション固有ロジックとブロックヘッダ中継のコア機能を分離する。ブロックヘッダ relay network は liveness のみで信頼され、zkSNARK correctness proof とともにヘッダを別チェーンの updater contract へ送る。受信チェーンのアプリケーションは updater contract から最新ヘッダを取得し、自身のロジックを実行する。

Telepathy は Ethereum と他チェーン間の任意メッセージパッシングを提供する。Ethereum block finality を示すため、validator signature の十分な割合を持つ block header を zkSNARK で証明し、destination chain の light client contract が検証する。その後 Merkle proof により transaction existence と finality を確認し、受信先 contract call を実行する。

6.5 Blockchain Storage

ブロックチェーンストレージでは、Merkle tree や Merkle Patricia tree によってデータの含有性と完全性を示す。しかし、巨大データセットに対する inclusion proof は大きくなり、オンチェーン検証コストが高くなる。ZKP は、データや計算をオフチェーンに置き、短い証明で正しさをオンチェーンに伝える手段となる。

Herodotus は storage proofs により、Ethereum の過去データをオンチェーンで検証可能にする。STARK proofs を用い、データ存在の inclusion proof と計算実行の proof を組み合わせる。Filecoin は zkSNARKs を大規模ストレージ証明に活用し、小さい証明と高速検証によりブロックチェーンストレージのスケラビリティを高める。

6.6 Smart Contract/Transaction Privacy

透明ブロックチェーン上で特定の dApp や transaction のみ選択的に秘匿する応用が、smart contract/transaction privacy である。proof generation は通常オフチェーンで行われ、proof が smart contract に提出され、検証後に状態更新や出金などの処理が行われる。

Tornado Cash は、deposit commitment の Merkle tree membership と nullifier check を zkSNARK で証明し、deposit address と withdrawal address のリンクを切る。匿名集合は deposit が増えるほど大きくなるが、タイミング相関や固定額 denomination などの制約がある。

Privacy Pools は、withdrawal が特定の deposit に紐づくことなく、ユーザが定義した association set に属することを証明する。これにより、flagged addresses を含まない集合からの資金であることを示しつつ、個別 transfer の unlinkability を維持しようとする。

Penumbra は Cosmos ecosystem 内の fully private proof-of-stake network と DEX であり、multi-asset shielded pool と zkSNARKs を使う。ZSwap は sealed-bid batch auctions と concentrated liquidity を組み合わせ、frontrunning を低減する。

Hawk は private smart contract framework で、public/private component を分割し、private portion を off-chain manager が実行して zkSNARK を生成する。Mina zkApps は user browser 上で smart contract code をローカル実行し、ZK-SNARK proof を生成して Mina network に提出する。

注意点: プライバシーと規制

匿名性の高いプロトコルは正当なプライバシー需要と不正利用の両方を引き寄せる。近年の研究は、選択的開示、規制対応 membership proof、multi-party authorization による追跡可能性など、プライバシーと説明責任の均衡を探っている。

6.7 Blockchain-Based Proof of Identity

ZKP は、身元や資格を直接明かさずに、グループメンバーシップや credential possession を証明できる。オンチェーン smart contract が privacy-preserving ZK identity proof を検証すれば、匿名投票、sybil-resistant reputation、access control などが可能になる。

Semaphore は Ethereum 上の zero-knowledge signaling framework である。ユーザは Pedersen commitment identity が Merkle tree の leaf として存在すること、秘密を知っていること、nullifier が未使用であること、signal が EdDSA signature により承認されていることを zkSNARK で証明する。World ID は Semaphore を proof-of-personhood に活用する。Galxe protocol は複数の identity commitments を一つの credential へまとめ、Poseidon hash、BN254、Groth16、nullifier を用いて selective proof を行う。Aleo の zPass は既存 ID documents から匿名 credential を作る。

6.8 Supply Chain/Enterprise Blockchain Privacy

サプライチェーンや企業ブロックチェーンでは、透明性、トレーサビリティ、改ざん耐性が求められる一方、価格、在庫、生産計画、取引条件などの機密情報を保護する必要がある。ZKP は、詳細データを隠しながら、製品真正性、コンプライアンス、属性、集計値などを証明する。

QEDIT は enterprise blockchains 向けの privacy-enhancing technology を提供し、取引内容を隠しながら妥当性を証明する。zk-BeSC は polynomial ZKPs を用いた supply chain framework で、Ethereum testnet 上の web3 application として実装され、検証ガスの削減を示す。Sahai et al. は Hyperledger Fabric と ZKP を用いて、traceability と privacy を両立する supply chain model を提示する。

表9の翻訳: エンタープライズ ZKP パターン

パターン	証明される statement	主ボトルネック	成熟度
Provenance tracking	製品 origin と chain of custody	hash-chain depth	pilot
Private compliance	private data 上の policy predicate	predicate circuit size	research
B2B attestation	レコードの aggregate properties	schema-dependent circuit	proof-of-concept
Selective disclosure	signed credential からの assertion	signature verification in-circuit	pilot

6.9 Proof of Reserves

Proof of Reserves (PoR) は、取引所や金融機関が、ユーザへの債務を十分にカバーする資産を保有していることを示す仕組みである。ZK-PoR は、具体的な準備金額、口座保有者、取引詳細を明かさずに solvency を検証可能にする。

典型的には、保有資産や負債を cryptographic commitment、Merkle tree、homomorphic encryption、elliptic curve encryption など表現し、準備金が十分であることを ZKP で示す。Provisions は Bitcoin exchange の solvency を privacy-preserving に証明する。Proven は企業や金融機関が liquidity や asset reserves を具体値なしに証明するプラットフォームである。

PoR の限界

ZK-PoR は数学的には強いが、liability completeness、point-in-time snapshot、address-to-reserve mapping、auditability と privacy のトレードオフという限界がある。つまり、証明された集合に全負債が含まれているか、証明後も資産が残っているか、証明対象アドレスが本当に当該機関のものは、別途監査や制度設計が必要である。

7. 原論文 第 5 章 Non-Blockchain Applications の翻訳と解説

7.1 非ブロックチェーン用の全像

表 10 の翻訳: 非ブロックチェーン ZKP 応用の概観

ユースケース	代表的研究/プロジェクト
ML/AI	zkCNN、vCNN、zkDL、Kaizen、zkLLM、ezkl、Modulus、Giza、TensorPlonk
Other applications	Proofs of Identity、zk-creds、voting ZK arguments、PhotoProof、VeeDo、Collaborative zkSNARKs、ZKP2P

7.2 Machine Learning and AI

7.2.1 動機と定義

AI 生成コンテンツが人間の作成物に近づくにつれ、特定のコンテンツが特定のモデルを特定の入力に適用して生成されたことを検証する必要がある。ZKML は、モデルや入力を秘匿しながら、推論や学習が正しく行われたことを証明する技術である。

たとえば、サービス提供者はモデルを公開せずに「この出力は指定モデルと入力から得られた」と証明できる。ユーザはモデルの詳細を知らなくても推論結果を信頼できる。医療や金融など、入力データやモデルパラメータが機密である領域で特に重要である。

7.2.2 方法論

典型的な ZKML では、サービス提供者が prover となり、所有する ML model を秘匿したまま、特定入力に対する推論や訓練が正しく実行されたことを verifier に示す。ML 計算は行列積、畳み込み、活性化関数、比較、量子化などを含み、これらを有限体上の制約へ落とす必要がある。

7.2.3 応用

vCNN は commit-and-prove を用いて convolutional neural networks を zkSNARK ベースで検証する。

zkCNN は sumcheck 技法により FFT と convolution を線形 prover time で証明し、評価設定では vCNN より大幅な高速化を報告する。

zkDL は deep learning training の ZKP を扱い、ReLU と backpropagation の非線形計算を効率的に検証するため zkReLU を導入する。FAC4DNN により、複数 layer と training step の proof を集約する。

Garg et al. は training proof の実験的検討を行い、強いセキュリティ保証と実用的オーバーヘッドの均衡を探る。Kaizen は gradient descent に最適化された sumcheck ベース証明を用い、iteration 数に対して proof size と verifier time が線形に増えないよう recursion で組み合わせる。

zkLLM は大規模言語モデルの reasoning result に対する ZKP を目指し、モデルやデータを隠したまま推論結果を検証可能にする。ezkl は高水準の ML computational graph から prover/verifier を実装するツールである。Modulus Labs は AI inference を ZK で検証する Remainder などを開発し、Giza は StarkNet 上で ONNX を活用した ML model deployment を目指す。

背景補足: なぜ ZKML は難しいか

ニューラルネットワークは浮動小数点、巨大行列、非線形活性化、比較、softmaxなどを多用する。一方、ZKP は有限体上の加算・乗算が得意で、ビット演算や非線形比較が重い。したがって、量子化、近似、lookup、sumcheck、専用回路、ハードウェア高速化の組み合わせが必要になる。

7.3 Other Non-Blockchain Applications

原論文は、ML/AI 以外の応用を、アイデンティティ、投票、メディア認証、ランダムネスとタイムロック、共同計算、Web2-Web3 payment bridging に分類する。共通する構造は「ルールに従ったことを、基礎データを明かさずに証明する」点である。

表 11 の翻訳: その他の非ブロックチェーン ZKP 応用

領域	何を証明するか	代表的研究/プロジェクト
Identity and credentials	credential や set membership の妥当性を秘匿して証明	Feige-Fiat-Shamir identification、zk-creds
Voting	投票内容を秘匿しつつ ballot validity と tally correctness を証明	Non-interactive ZK arguments for voting
Media authentication	画像が許可された編集範囲で変換されたことを証明	PhotoProof
Randomness and timelocks	delay function と派生 randomness の正しさを証明	VeeDo
Collaborative computation	複数者の秘密入力上で正しい共同計算が行われたことを証明	Collaborative zkSNARKs
Web2-Web3 payment bridging	メール内容を明かさず DKIM 署名など off-chain payment evidence の妥当性を証明	ZKP2P

zk-creds は既存 ID documents から匿名 credential を構築し、issuer が signing keys を維持しなくても検証可能にする。PhotoProof は、画像が許可された編集のもとで正しい変換を受けたことを暗号的に認証する。VeeDo は STARK ベースの VDF service でタイムロックや randomness に関係する。Collaborative zkSNARKs は、入力を共有せずに joint analytics を行う。ZKP2P は DKIM 署名付き payment email の性質を証明し、Web2 payment rails と on-chain settlement をつなぐ。

8. 原論文 第 6 章 Conclusion and Future Work の翻訳と解説

8.1 結論の翻訳

ゼロ知識証明は、理論的構成から、暗号研究と実世界デプロイを橋渡しする実用的ツールへ成熟した。本サーベイは、ブロックチェーンプライバシー、スケーラブル計算、アイデンティティ、検証可能機械学習、規制対応といった新興の非ブロックチェーン領域における応用を検討した。

進歩は速いが、証明システム最適化、開発者ツール、ブロックチェーン外への普及には大きな課題が残る。特に有望なのは、ZKP とゲーム理論的メカニズムの統合である。たとえば、資産情報を明かさずに真実なオークション入札を可能にすること、プライバシー保護型の金融価格発見、private order-book exchanges、AMM モデルを用いた dark pools、MEV を緩和する encrypted transaction mempools などが研究フロンティアである。

8.2 今後の究方向

8.2.1 Standardization and interoperability

ZKProof.org のような取り組みは、用語とセキュリティガイドラインの標準化を進めている。しかし、共通中間表現や普遍的 proof formats はまだ十分ではない。proof systems 間の interoperability standard と再現可能な benchmark suite が整えば、vendor lock-in を下げ、実務導入が進む。

8.2.2 Post-quantum considerations

STARKs や FRI ベース protocol は、量子攻撃に弱い pairing-based assumptions を避けられる。一方、lattice-based や hash-based SNARKs も研究されている。問題は、量子耐性と proof size、オンチェーン検証コストとのバランスである。

8.2.3 Circuit security and verification tooling

under-constrained circuits や手書きバグは重大な実務リスクである。constraint system の形式検証、ZK DSL 向け static analysis、自動 under-constraint detection が必要である。理論上安全な証明方式でも、回路が仕様を正しく表していなければ安全ではない。

8.2.4 Performance roadmap

proof recursion、composition、Nova などの folding scheme、IVC、FPGA/ASIC 高速化は、低遅延 proving へ向けた道筋である。prover cost が数桁下がれば、L2 rollups 間の同期的 composability や大規模 ZKML など、現在は証明生成時間がボトルネックであるユースケースが開ける。

総括

ZKP の次の課題は、単に「証明できる」から「安く、速く、安全に、標準化された形で、開発者が間違えずに証明できる」へ移行することである。

9. 横断的な設計論点、限界、批判的読解

9.1 証明方式を選ぶときの実務チェックリスト

- 秘匿したいのは witness か、計算過程か、モデルか、取引履歴か。
- 必要なのは zero-knowledge privacy か、それとも validity proof だけか。
- 証明はオンチェーンで検証するか、オフチェーンで検証するか。
- trusted setup を制度上・運用上許容できるか。
- 証明生成はユーザ端末、サーバ、分散 prover network、専用ハードウェアのどこで行うか。
- データ可用性は L1 calldata/blob、off-chain DA、validium、volition のどれか。
- 回路や DSL の監査体制はあるか。under-constrained circuit を検出できるか。
- 長期安全性、ポスト量子性、鍵/検証鍵更新、回路アップグレードの方針はあるか。

9.2 応用別に見た「ZKP が効く理由」

応用別の ZKP 価値

応用	ZKP が解く問題	主な制約
L2 scaling	L1 が全計算を再実行せず状態遷移を検証できる	proof generation、DA cost、sequencer decentralization
transaction privacy	取引内容を秘匿しつつ二重支出や残高保存を証明	匿名集合、timing analysis、規制対応
identity	資格や membership を秘匿して証明	credential issuer、nullifier、Sybil resistance
storage proof	巨大データの含有性や履歴を短く検証	データ可用性、historical data access
Proof of Reserves	準備金十分性を秘匿して証明	負債網羅性、snapshot 問題、第三者監査
ZKML	モデル/入力を隠して推論・学習を検証	浮動小数点、非線形、巨大モデル、prover time

9.3 原論文を読む際の注意点

第一に、原論文はサーベイであるため、個別プロジェクトの最新状態や性能値は変化しうる。実務導入では、各プロジェクトの現行仕様、監査状況、ベンチマーク、メインネット実績を追加確認する必要がある。

第二に、表の「trusted」「transparent」「post-quantum」などは大まかな傾向であり、証明方式の具体的なインスタンス、ハッシュ、曲線、PCS、再帰構成により異なる。

第三に、ZKP は秘密情報を「証明から」漏らさないが、メタデータ、タイミング、ネットワーク観測、UX の設計ミス、集合サイズの小ささから情報が漏れる可能性がある。暗号プロトコルと運用システムは区別して評価すべきである。

第四に、証明方式の安全性とアプリケーションの安全性は別である。たとえば proof system が sound でも、回路が under-constrained なら不正 witness が通る。PoR で proof が正しくても、負債集合から口座を除外すれば solvency を誤認させられる。

9.4 研究課題の整理

- 証明生成の高速化: MSM、NTT、hashing、lookup、folding、hardware acceleration。
- 開発者安全性: DSL、型システム、形式検証、監査可能な constraint IR。
- 標準化: proof format、benchmark、security definition、interoperability。

- 規制対応プライバシー: selective disclosure、association sets、accountable privacy。
- ZKML: 量子化、非線形関数、巨大モデルの分割証明、推論結果の provenance。
- UX: wallet/key handling、proof latency、client-side proving、mobile proving。

10. 付録 A: 用語集

AIR: Algebraic Intermediate Representation。実行トレースに対する多項式制約で計算を表す。STARK 系で多い。

Argument: 計算量仮定に基づく証明体系。無制限の不正者ではなく効率的な不正者に対して健全性を持つ。

Bulletproofs: trusted setup 不要の短い range proof 系。SNARK/STARK ほど簡潔ではない傾向。

CRS: Common Reference String。証明者・検証者が共有する公開パラメータ。trusted setup で生成されることがある。

DA: Data Availability。rollup など、状態を再構成するためのデータが利用可能であること。

Fiat-Shamir: 対話型証明のランダムチャレンジをハッシュで置き換える非対話化手法。

Folding scheme: 複数の計算ステップや証明を逐次的に折り畳む構成。Nova など。

FRI: Fast Reed-Solomon IOP of Proximity。STARK で低次数性検査に使われる。

Groth16: 非常に短い証明と高速検証で知られる pairing-based zkSNARK。回路ごとの trusted setup が課題。

IVC: Incrementally Verifiable Computation。長い逐次計算を段階的に証明し、検証コストを抑える。

Lookup argument: 表引きで制約を効率化する技法。range check やハッシュ、非ネイティブ演算で有用。

MSM: Multi-Scalar Multiplication。SNARK proving の主要ボトルネック。

NTT: Number Theoretic Transform。有限体上の FFT。多項式演算を高速化する。

Nullifier: 匿名集合内の二重使用を防ぐための公開値。同じ秘密から一意に導出されることが多い。

PCS: Polynomial Commitment Scheme。多項式全体にコミットし、特定点での評価を短く開示する。

Plonkish: PLONK 由来の custom gates、permutation、lookup を持つ算術化の総称。

QAP: Quadratic Arithmetic Program。R1CS を多項式関係へ変換する表現。

R1CS: Rank-1 Constraint System。線形結合の積が線形結合に等しい形で制約を表す。

STARK: Scalable Transparent Argument of Knowledge。transparent setup と post-quantum security が特徴。ただし証明は大きめ。

Trusted setup: 秘密 trapdoor を含むセットアップ。漏えいすると安全性に影響しうる。

Witness: statement を真にする秘密データや実行トレース。

zkEVM: EVM 実行を ZK で証明可能にする実行環境。EVM compatibility/equivalence の程度は方式により異なる。

zkML/ZKML: 機械学習の推論・学習をゼロ知識または検証可能計算として証明する分野。

zkSNARK: Zero-Knowledge Succinct Non-Interactive Argument of Knowledge。短く非対話で、ゼロ知識性を持つ。

zkSTARK: Zero-Knowledge STARK。transparent setup、post-quantum secure 傾向、ハッシュベース。

11. 付録 B: 図表の一覧と読み取りポイント

図一覧

図	原論文ページ	内容	読み取りポイント
図 1	p.3	ZKP 応用と開発ツール	開発基盤と応用領域の関係を示す全体地図
図 2	p.5	ZKP パイプライン	高水準プログラムから多項式証

図	原論文ページ	内容	読み取りポイント
			明へ変換する流れ
図 3	p.7	zkVM アーキテクチャ	プログラム実行を証明する VM の入力・内部・出力
図 4	p.9	zkDSL 処理	コードから制約、証明システム、証明への流れ
図 5	p.10	暗号コンポーネント分類	ガジェット、証明方式、楕円曲線の部品群
図 6	p.13	L1 のデータ圧縮とプライバシー	ZKP がスケーリングと機密性に効く二つの構造
図 7	p.17	zkBridge 構成	ブロックヘッダ証明を使ったクロスチェーン転送

表一覧

表	内容	役割
表 1	証明ファミリの傾向	ZKP/SNARK/zkSNARK/STARK/Bulletproofs の違いを俯瞰
表 2	算術化戦略	R1CS、Plonkish、AIR、CCS の比較
表 3	zkVM 設計選択	RISC Zero、CairoVM、Miden VM の比較
表 4	zkDSL 比較	言語、証明システム、中間表現、用途の対応
表 5	楕円曲線サポート	主要ライブラリと曲線の対応
表 6	ハードウェア高速化	CPU/GPU/FPGA/ASIC の向き不向き
表 7	ブロックチェーン応用	応用カテゴリと代表プロジェクト
表 8	L2 validity systems	proof family、EVM レベル、DA モードなど
表 9	エンタープライズパターン	証明される statement とボトルネック
表 10	非ブロックチェーン応用	ML/AI とその他応用の概観
表 11	その他非ブロックチェーン応用	identity、voting、media、timelock など

12. 付録 c: 参考文献ガイド（主要カテゴリ別）

原論文には 111 件の参考文献が挙げられている。本書では、本文理解のために、代表的な文献群をカテゴリ別に整理する。番号は原論文の参考文献番号に対応する。

参考文献カテゴリガイド

カテゴリ	主な番号	内容
ZKP の古典的基礎	[1]-[6], [15], [18], [19]	対話型証明、ゼロ知識、SNARK、QAP、Fiat-Shamir、PCS
ZKP サーベイ/背景	[11]-[13]	zkSNARK 解説、range proof survey、blockchain ZKP survey
zkVM/DSL/ライブラリ	[20]-[59]	Circom、RISC Zero、Cairo、Miden、Lurk、Leo、Arkworks、Gnark、Halo2、Nova、hardware acceleration
ブロックチェーン L1/L2	[60]-[70]	Zcash、Aleo、Mina、Ethereum zk-rollups、zkSync、Polygon zkEVM、

カテゴリ	主な番号	内容
		Scroll、Linea、Starknet、Aztec
相互運用性・ストレージ	[71]-[75]	zkBridge、Telepathy、Herodotus、STARK、Filecoin
プライバシー・アイデンティティ	[76]-[85]	Privacy Pools、Platypus、accountable privacy、Penumbra、private DAO、Mina zkApps、Semaphore、World ID、Galxe、zPass
エンタープライズ・PoR	[86]-[93]	QEDIT、zk-BeSC、supply chain、zkLedger、zk-creds、ZK-PoR、Provisions、Proven
ZKML/AI	[94]-[105]	GPT-4、vCNN、zkCNN、zkDL、Kaizen、zkLLM、ezkl、Modulus、Giza
その他応用	[106]-[111]	proof of identity、voting、PhotoProof、VeeDo、collaborative zkSNARKs、ZKP2P

13. 付録 D: 実務者向け最終メモ

ZKP 導入の可否は、暗号方式だけでなく、プロダクト要件、脅威モデル、UX、監査可能性、運用、ハードウェア、データ可用性、規制環境で決まる。

- ZK を使う前に、公開したくない情報と検証したい predicate を明確にする。
- 「ZK である」ことと「安全なアプリケーションである」ことを混同しない。
- 専用回路は効率が高いが開発負荷が高い。zkVM は開発しやすいが proving cost が上がりやすい。
- オンチェーン検証では proof size と verifier gas が支配的である。オフチェーン検証では prover cost とメモリが支配的になりやすい。
- ZKML は有望だが、大規模 LLM の完全証明は依然として非常に重い。小規模モデル、量子化モデル、特定演算の証明から実用化が進む。
- 今後の鍵は、標準化、監査可能な DSL、proof recursion/folding、ハードウェア高速化、そして規制対応プライバシーである。